Institut Supérieur d'Informatique et de Mathématiques de Monastir ISIMM

DS – S1 – 2026/2027

Filière : *L3 INFO*	**Matière : Développement Mobile**		**Enseignante : Dorra DHAOU**
Date : 09/ 11 / 2026	Nbr de Crédits : 3	Coefficient : **1.5**	**Documents autorisés : Non**
Durée de l'examen : 1h00	Régime d'évaluation : **Mixte** DS		**Nombre de pages : 07**

Exercice 1 : QCM (6 points)

Choisissez-la (ou les) bonne(s) réponse(s) parmi les propositions suivantes. Inscrivez vos réponses sous forme d'un tableau, comme illustré ci-dessous :

1	2	3	4	5	6	7	8	9	10	11	12

1. **Quel élément de l'architecture Android gère les processus, la mémoire et les pilotes matériels ?**
   a. Le noyau Linux
   b. Les applications utilisateur
   c. Le fichier AndroidManifest.xml
   d. Le gestionnaire de ressources XML.
2. **Quel rôle joue ART dans Android ?**
   a. Décrire les interfaces graphiques en XML
   b. Remplacer le système de fichiers
   c. Fournir l'environnement d'exécution des applications Android
   d. Créer automatiquement les fichiers de ressources
3. **Une View possède l'identifiant btnValidate dans un layout chargé. Quelle instruction permet de la retrouver avec l'approche classique ?**
   a. findViewById(R.layout.btnValidate)
   b. findViewById(R.id.btnValidate)
   c. R.string.btnValidate
   d. getResource(btnValidate)
4. **On considère le fichier XML suivant. Quelles affirmations sont correctes ?**

```
<TextView
android:id="@+id/txtNom"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="@string/nom" />
```

   a. `@+id/txtNom` crée un nouvel identifiant.
   b. `@id/txtNom` crée systématiquement un nouvel identifiant.
   c. `@string/nom` fait référence à une ressource de type chaîne.
   d. `wrap_content` impose une taille fixe de 100dp.
   e. L'identifiant peut être utilisé avec `R.id.txtNom`.

5. **Une Activity visible perd le premier plan au profit d'une autre Activity. Quel callback intervient généralement en premier ?**
   a. onPause()
   b. onDestroy()
   c. onCreate()
   d. onRestart()

6. **Une Activity arrêtée mais non détruite revient au premier plan. Quelle séquence est habituelle ?**
   a. onCreate() → onDestroy() → onResume()
   b. onRestart() → onStart() → onResume()
   c. onResume() → onCreate() → onStop()
   d. onDestroy() → onRestart() → onPause()

7. **À propos de ViewBinding, quelles affirmations sont correctes ?**
   a. Il permet d'accéder aux vues du layout sans appeler systématiquement findViewById.
   b. Il est généré à partir des fichiers XML de layout.
   c. Il supprime la nécessité de définir les identifiants des vues.
   d. Il garantit que toute vue sera toujours non nulle, quel que soit le layout chargé.
   e. Il peut réduire les erreurs liées aux mauvais identifiants de vues.

8. **Quel mécanisme convient pour sauvegarder un petit état temporaire lors d'une recréation ?**
   a. Un Toast
   b. onSaveInstanceState() et la restauration de l'état
   c. Une nouvelle permission dans le manifeste
   d. setOnClickListener()

9. **Dans une pile d'activités A → B → C, C étant au premier plan, que fait généralement le bouton Retour ?**
   a. Il affiche B en retirant C de la pile
   b. Il affiche directement A sans autre action
   c. Il redémarre C
   d. Il supprime systématiquement toute l'application

10. On considère ce layout :

```
<Button
    android:id="@+id/btn"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Valider" />
```

Quelle affirmation est correcte ?
   a. `wrap_content` adapte la dimension au contenu.
   b. `wrap_content` occupe toujours tout l'écran.
   c. `match_parent` adapte la dimension uniquement au texte.
   d. Les deux valeurs sont équivalentes.

11. On considère le code Kotlin suivant :

```
val btn = findViewById<Button>(R.id.btnCalcul)

btn.setOnClickListener {
    Toast.makeText(this, "A", Toast.LENGTH_SHORT).show()
}

btn.setOnClickListener {
    Toast.makeText(this, "B", Toast.LENGTH_SHORT).show()
}
```

Quelle sera la conséquence lorsqu'un utilisateur clique sur le bouton ?

a. Les messages « A » et « B » seront affichés successivement.
b. Seul le message « A » sera affiché, car le premier listener est prioritaire.
c. Seul le message « B » sera affiché, car le deuxième listener remplace le premier.
d. Aucun message ne sera affiché, car un bouton ne peut pas avoir deux listeners.

12. On exécute le code suivant :

```
var compteur = 0

btnIncrementer.setOnClickListener {
    compteur++
    txtCompteur.text = compteur.toString()
}
```

L'utilisateur clique trois fois sur le bouton, puis fait pivoter l'écran. Si l'Activity est recréée et que le code est réinitialisé, quelle affirmation est correcte ?

a. Le compteur vaut automatiquement 3 dans la nouvelle instance.
b. Le compteur revient à 0 si aucune sauvegarde ou restauration n'est prévue.
c. Android sauvegarde automatiquement toutes les variables Kotlin.
d. La rotation supprime définitivement le bouton du layout.

Exercice 2 : Questions du Cours (4 points)

1. Pourquoi est-il préférable de définir une chaîne dans strings.xml plutôt que directement dans le code Kotlin ?
2. Citer deux avantages de l'utilisation d'un émulateur Android.
3. Si le fichier activity_main.xml est renommé accueil.xml, quelle référence faut-il utiliser pour charger ce layout ?
4. Quelle est la différence entre match_parent et wrap_content ?
5. À quel moment la méthode onSaveInstanceState(Bundle b) est-elle appelée et quel est son rôle ?
6. Quel est le rôle de l'objet **Bundle** dans le cycle de vie d'une activité Android ?
7. Une application Android contient un champ de saisie EditText permettant à l'utilisateur de saisir une valeur, ainsi qu'une variable entière compteur déclarée dans le code Kotlin.

   Après une rotation de l'écran, on constate que le contenu du champ EditText est conservé, tandis que la valeur de la variable compteur est perdue.

   a. Expliquer pourquoi le contenu du EditText est conservé alors que la valeur de compteur est perdue.
   b. Quelles modifications faut-il apporter au code pour garantir la conservation et la restauration de ces deux valeurs après une rotation de l'écran ?

Exercice 3 : (10 points)

On souhaite développer une application Android permettant à un étudiant de vérifier s'il a réussi son semestre à partir de trois notes : Algorithmique, Programmation Web et Bases de données.

L'application est composée de deux activités :

- **MainActivity** : permet de saisir les trois notes et de calculer la moyenne.
- **ResultActivity** : affiche un message de bienvenue et une image illustrant le résultat universitaire.

Règles de calcul :

$$\text{Moyenne} = (\text{Note1} + \text{Note2} + \text{Note3}) / 3$$

- Si la moyenne est supérieure ou égale à 10 : afficher « Semestre validé ».
- Sinon : afficher « Semestre non validé ».

Question 1 – Gestion des ressources Android

1.1. Compléter le fichier `strings.xml` avec les ressources suivantes :

- Nom de l'application : « University Results ».
- Titre de l'activité : « Résultat universitaire ».
- Message de succès : « Félicitations ! Semestre validé. ».
- Message d'échec : « Semestre non validé. ».

```
<(1)???>
    <string name="app_name"> (2)???</string>
    <string name="title_main"> (3)???</string>
    <string name="success_message"> (4)???</string>
    <string name="failure_message"> (5)???</string>
</(1) ???>
```

1.2. Écrire le fichier `colors.xml` afin de définir les couleurs suivantes :

- `blue` : `#2196F3`.
- `green` : `#4CAF50`.
- `red` : `#F44336`.

1.3. Deux images sont fournies dans le projet :

- `success.png` : image représentant la réussite.
- `failure.png` : image représentant l'échec.

a. Indiquer le dossier dans lequel ces images doivent être placées.
b. Compléter les deux références suivantes :

```
<ImageView
    android:id="@+id/imgSuccess"
    ...
    android:src= (6)??? />

<ImageView
    android:id="@+id/imgFailure"
    ...
    android:src= (7)??? />
```

Question 2 – MainActivity : Calcul et validation

Dans `MainActivity`, l'étudiant saisit trois notes et clique sur le bouton « Vérifier ».

L'application doit :

1. Récupérer les trois notes et vérifier que les champs ne sont pas vides.
2. Vérifier que chaque note est comprise entre 0 et 20 et Calculer la moyenne.

3. Afficher un Toast :
    - « Semestre validé » si la moyenne est supérieure ou égale à 10.
    - « Semestre non validé » sinon.
4. Lancer `ResultActivity`.

Travail demandé : Compléter le code suivant.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        (1) ???  // chargement du layout

        // Récupération des vues à partir de son identifiant
        val etNote1 = (2)???
        val etNote2 = (3)???
        val etNote3 = (4)???
        val btnVerify = (5)???

        (6)??? { // gérer le clic sur le bouton btnVerify

            // Récupérer le contenu saisi dans le champ EditText.
            val s1 = etNote1(7)???.toString()
            val s2 = etNote2.(7)???.toString()
            val s3 = etNote3.(7)???.toString()

            if (s1.isBlank() || s2.isBlank() || s3.isBlank()) {
                (8)???(this, "Saisie obligatoire", Toast.LENGTH_SHORT).show()
            }
            else {
                val n1 = s1.toDouble()
                val n2 = s2.toDouble()
                val n3 = s3.toDouble()

                // vérifier si au moins une note est inférieure à 0 ou supérieure à 20.
                if (9)??? {

                    (8)???(this, "Notes invalides", Toast.LENGTH_SHORT).show()
                }
                else {

                    val moyenne = (10)??? // calculer la moyenne des trois notes.

                    if ((11) ???) {

                        (8)???(this, getString((12) ???), Toast.LENGTH_SHORT).show()

                    }
                    else {
                        (8)???(this, getString((13)???), Toast.LENGTH_SHORT).show()
                    }
                    // Navigation vers la 2ème activité
                    val intent = (14) ???

                    (15)???

                }
            }
        }
    }
}
```

Question 3 – ResultActivity : Affichage et navigation

Dans `ResultActivity`, on souhaite afficher un message de bienvenue ainsi qu'un bouton « Retour ».

L'image affichée est `success.png` par défaut.

Travail demandé :

3.1. Compléter le code suivant pour charger le layout et récupérer les vues.

3.2. Afficher le message « Bienvenue dans votre espace étudiant ».

3.3. Gérer le clic sur le bouton « Retour » pour revenir à `MainActivity`.

```
class ResultActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        (1) ???      //chargement du layout

        // Récupération des vues à partir des identifiants
        val tvWelcome = (2)???
        val imgResult = (3)???
        val btnBack = (4)???

        // modifier le texte du TextView tvWelcome.
        tvWelcome.(5)??? = getString((6) ???)

        // Gestion du clic sur Retour
        (7)??? {
            finish()
        }
    }
}
```

Annexe – Extraits XML

Fichier `activity_main.xml`

```
<LinearLayout
    ...
    android:orientation="vertical"
    ...>

    <EditText
        android:id="@+id/etNote1"
        ...
        android:inputType="numberDecimal"
        android:hint="Note d'Algorithmique" />

<EditText
    android:id="@+id/etNote2"
    ...
    android:inputType="numberDecimal"
    android:hint="Note de Programmation Web" />

<EditText
    android:id="@+id/etNote3"
    ...
    android:inputType="numberDecimal"
    android:hint="Note de Bases de données" />

    <Button
        android:id="@+id/btnVerify"
        ...
        android:text="Vérifier" />

</LinearLayout>
```

Fichier `activity_result.xml`

```
<LinearLayout
    ...
    android:orientation="vertical"
    ...>

    <TextView
        android:id="@+id/tvWelcome"
        ... />

    <ImageView
        android:id="@+id/imgResult"
        ... />

    <Button
        android:id="@+id/btnBack"
        ...
        android:text="Retour" />

</LinearLayout>
```

BON TRAVAIL